

Appendix 1: Illustrative Example of CLEAR

This appendix provides a small numerical example to clarify how CLEAR transforms raw HDFS observations into category-aware replication and locality decisions. The example is synthetic; its purpose is not to report experimental performance, but to make the successive computations easier to follow.

We consider eight files $f_1, \dots, f_8$ stored in HDFS and observed during one rolling window P . During this window, the Log Ingestor and the Feature Builder collect the raw counters needed to compute the five signals used by the controller: access frequency, file age, write ratio, locality, and peak concurrency. Table 1 summarizes these counters.

Table 1: Raw usage counters over one window P

File	Total accesses	Writes	Age (days)	Local accesses	Remote accesses	Peak conc
f_1	900	160	2	780	120	15
f_2	800	120	3	700	100	12
f_3	600	80	10	180	420	10
f_4	550	40	12	150	400	9
f_5	320	32	20	230	90	5
f_6	300	24	25	225	75	4
f_7	40	8	90	38	2	1
f_8	30	16	120	28	2	1

0.0.1 Step 1: From Raw Logs to Normalized Features

The Feature builder converts these counters into the five features used by the clustering step: access frequency, file age, write ratio, locality, and peak concurrency. In this toy window, the access frequency $freq(f_i)$ is simply the total number of accesses in P , the age $age(f_i)$ is measured in days since creation, and the locality is the fraction of accesses served from a local replica. The global mean number of writes across all files is

$$\mu_{\text{writes}} = \frac{1}{8} (160 + 120 + 80 + 40 + 32 + 24 + 8 + 16) = 60,$$

so the write ratio of each file is obtained by dividing its write count by this mean,

$$wRatio(f_i) = \frac{writes(f_i)}{\mu_{\text{writes}}}.$$

Table 2 reports the resulting feature values before normalization.

Table 2: Five per file features before normalization

File	$freq(f_i)$	$age(f_i)$ (days)	$wRatio(f_i)$	$locality(f_i)$	$conc(f_i)$
f_1	900	2	2.67	0.87	15
f_2	800	3	2.00	0.88	12
f_3	600	10	1.33	0.30	10
f_4	550	12	0.67	0.27	9
f_5	320	20	0.53	0.72	5
f_6	300	25	0.40	0.75	4
f_7	40	90	0.13	0.95	1
f_8	30	120	0.27	0.93	1

The Feature scaler then applies the min-max normalization defined earlier, feature by feature, to map all five dimensions to the interval $[0, 1]$.

Table 3 shows the resulting normalized feature vectors $\mathbf{x}'_i$ for the eight files.

Table 3: normalized features $\mathbf{x}'_i$ in $[0, 1]^5$

File	$freq'$	age'	$wRatio'$	$locality'$	$conc'$
f_1	1.000	0.000	1.000	0.877	1.000
f_2	0.885	0.008	0.737	0.889	0.786
f_3	0.655	0.068	0.474	0.040	0.643
f_4	0.598	0.085	0.211	0.000	0.571
f_5	0.333	0.153	0.158	0.659	0.286
f_6	0.310	0.195	0.105	0.705	0.214
f_7	0.011	0.746	0.000	1.000	0.000
f_8	0.000	1.000	0.053	0.975	0.000

0.0.2 Step 2: Clustering in the Feature Space

The X-Means++ engine runs on the normalized dataset and starts from $K_{\min} = 4$ clusters, with a cap $K_{\max}$ that remains small. In this toy example, the split proposals lead to a final partition with four clusters:

$$C_1 = \{f_1, f_2\}, \quad C_2 = \{f_3, f_4\}, \quad C_3 = \{f_5, f_6\}, \quad C_4 = \{f_7, f_8\}.$$

Files f_1 and f_2 have very high frequency, high write ratios, and strong concurrency. Files f_3 and f_4 still see sizeable demand and concurrency but suffer from low locality because a large share of their accesses are remote. Files f_5 and f_6 sit closer to the centre of the workload, without extreme values on any feature. Finally, f_7 and f_8 are clearly older, rarely accessed, and almost never updated.

Table 4 gives a compact view of the four clusters and their qualitative behavior.

Table 4: Clusters and qualitative behavior

Cluster	Files	Qualitative profile
C_1	$\{f_1, f_2\}$	Very high frequency, high write ratio, high concurrency
C_2	$\{f_3, f_4\}$	Medium to high frequency, high concurrency, low locality
C_3	$\{f_5, f_6\}$	Medium frequency and concurrency, close to global centre
C_4	$\{f_7, f_8\}$	Old files, very low frequency and concurrency, near idle

0.0.3 Step 3: Category Scores, Medians, and Labels

The Category Mapper now uses the normalized features to compute, for each feature p , a global median $M_{\text{global},p}$ and a global median absolute deviation MAD_p on the current window.

On the toy dataset of Table 3, the global medians and MAD values are:

Table 5: Global medians and MAD scales per feature

Feature p	$M_{\text{global},p}$	MAD_p
freq'	0.466	0.305
age'	0.119	0.094
wRatio'	0.185	0.158
$\text{locality}'$	0.791	0.158
conc'	0.429	0.286

For one feature, the computation can be written explicitly. Consider the normalized write ratios

$$\text{wRatio}'(f_i) \in \{1.000, 0.737, 0.474, 0.211, 0.158, 0.105, 0.000, 0.053\}.$$

Sorting these values gives

$$0.000, 0.053, 0.105, 0.158, 0.211, 0.474, 0.737, 1.000.$$

With eight points, the global median is the average of the fourth and fifth values:

$$M_{\text{global},\text{wRatio}} = \frac{0.158 + 0.211}{2} = 0.184.$$

The MAD is the median of the absolute deviations $|\text{wRatio}'(f_i) - M_{\text{global},\text{wRatio}}|$. In this toy example, these deviations are

$$\begin{aligned} |1.000 - 0.184| &= 0.816, & |0.737 - 0.184| &= 0.553, & |0.474 - 0.184| &= 0.290, & |0.211 - 0.184| &= 0.027, \\ |0.158 - 0.184| &= 0.026, & |0.105 - 0.184| &= 0.079, & |0.000 - 0.184| &= 0.184, & |0.053 - 0.184| &= 0.131. \end{aligned}$$

Sorting these absolute deviations gives

$$0.026, 0.027, 0.079, 0.131, 0.184, 0.290, 0.553, 0.816.$$

With eight values, the MAD is the average of the fourth and fifth elements:

$$\text{MAD}_{\text{wRatio}} = \frac{0.131 + 0.184}{2} = 0.158,$$

which is consistent with the value reported in Table 5.

For each cluster C_k and feature p , the Category Mapper then computes the cluster median $M_{k,p}$, the signed deviation $\Delta_{k,p}$, and the standardized gap $z_{k,p}$.

Table 6 summarizes the standardized gaps for all clusters and all features, rounded to two decimals.

Table 6: Toy example: standardized gaps $z_{k,p}$ per cluster and feature

Cluster	freq'	age'	wRatio'	$\text{locality}'$	conc'
C_1	+1.57	-1.23	+4.33	+0.58	+1.62
C_2	+0.53	-0.45	+1.00	-4.88	+0.62
C_3	-0.47	+0.59	-0.34	-0.69	-0.62
C_4	-1.51	+8.06	-1.00	+1.24	-1.50

For instance, for cluster $C_1 = \{f_1, f_2\}$ the cluster median for the normalized write ratio is

$$M_{1,wRatio} = \frac{1.000 + 0.737}{2} = 0.868,$$

so the signed deviation and standardized gap on this feature are

$$\Delta_{1,wRatio} = 0.868 - 0.184 = 0.684, \quad z_{1,wRatio} = \frac{0.684}{0.158} = 4.33,$$

which shows that cluster C_1 sits far above the global level on the write ratio dimension.

Directional category scores:

The directional categories (*Critical Hot Data*, *Shared Multi-User Data*, *Archival Data*) use the standardized gaps $z_{k,p}$ and only keep contributions that follow the expected direction for each feature.

For *Critical Hot Data*, the feature set is $F_{\text{hot}} = \{freq, wRatio, conc\}$ and all three features are expected above the global median. With equal weights $w(p, \text{hot}) = 1/3$, the score of a cluster C_k is

$$S_{\text{hot}}^{(k)} = \frac{1}{3} ([z_{k,freq}]_+ + [z_{k,wRatio}]_+ + [z_{k,conc}]_+), \quad \text{where } [\cdot]_+ = \max(\cdot, 0).$$

For cluster C_1 , all three gaps are positive and the score is

$$S_{\text{hot}}^{(1)} = \frac{1}{3} (1.57 + 4.33 + 1.62) = 2.51,$$

which reflects a strong match with the *Critical Hot Data* story.

Moderate proximity scores:

In the toy example we set $\tau_{\text{moderate}} = 1.2$ and $\tau_{\text{ext}} = 3$, which means that Moderate clusters must stay within about 1.2 MAD of the global median on both access frequency and concurrency, and within 3 MAD on all features.

Table 7 shows the per feature components $h_{k,p}$ for frequency and concurrency. Values are rounded to two decimals.

Table 7: Moderate proximity components per cluster

Cluster	$h_{k,freq}$	$h_{k,conc}$
C_1	0.00	0.00
C_2	0.56	0.48
C_3	0.61	0.48
C_4	0.00	0.00

Clusters C_2 and C_3 have similar base proximity scores, since both are relatively close to the global medians on frequency and concurrency. However, C_2 is rejected by the second constraint because it is very extreme on locality (large negative $z_{2,locality}$), while C_3 does not show any such extreme deviation. As a result only C_3 keeps a non zero Moderate score.

Summary of category scores and labels:

Plugging the values of Table 6 and the Moderate definition above into the expressions for the four categories gives the scores reported in Table 8. Values are rounded to two decimals.

Table 8: Category scores per cluster

Cluster	$S_{\text{hot}}^{(k)}$	$S_{\text{shared}}^{(k)}$	$S_{\text{archival}}^{(k)}$	$S_{\text{moderate}}^{(k)}$
C_1	2.51	1.06	0.00	0.00
C_2	0.72	2.01	0.00	0.00
C_3	0.00	0.23	0.51	0.55
C_4	0.00	0.00	3.02	0.00

0.0.4 Step 4: Replication Factors per Cluster

The Replication Factor Decision Module now turns the qualitative labels and confidence scores into concrete protection levels. For illustration, we assume the bounds reported in Table 9, with *Archival* using erasure coding as baseline when $R_{\min}(\text{Archival}) = 0$.

Table 9: Replication bounds per category

Category	$R_{\min}(c)$	$R_{\max}(c)$
<i>Hot</i>	3	4
<i>Shared</i>	3	3
<i>Moderate</i>	2	3
<i>Archival</i>	0	1

For each category c , the module computes the median confidence med_c and the median absolute deviation MAD_c across all clusters that chose this category. Clusters whose confidence lies clearly above the typical level for their category are treated as strong representatives of that behavior and receive one extra replica within the allowed bounds. Clusters that are closer to the typical level stay at the minimum replication level for their category.

Applying the rule on the toy example yields the per-cluster decisions reported in Table 10.

Table 10: Category, Strong fit flag, and replication decision per cluster

Cluster	Files	Assigned category	Strong fit	Replication decision
C_1	$\{f_1, f_2\}$	Critical Hot Data	true	$R_1 = 4$ full replicas
C_2	$\{f_3, f_4\}$	Shared Multi-User Data	false	$R_2 = 3$ full replicas
C_3	$\{f_5, f_6\}$	Moderately Accessed Data	false	$R_3 = 2$ full replicas
C_4	$\{f_7, f_8\}$	Archival Data	true	$R_4 = 1$ full replica + RS(6,3)

0.0.5 Step 5: Locality Driven Relocation for a Shared File

The last part of the workflow concerns the Locality Manager, which only acts on files labeled as *Shared Multi-User Data*. In this toy example, we focus on file f_3 , which belongs to cluster C_2 . We assume that the replication factor of f_3 is three and that its replicas are currently stored on three DataNodes n_1 , n_2 , and n_3 .

During the same window P , the system maintains per node access counters $d_{f_3}(n)$ that count how many requests for f_3 come from each DataNode, regardless of which replica serves them. A possible distribution that is consistent with $\text{freq}(f_3) = 600$ is shown in Table 11.

Table 11: Per node access counters for shared file f_3

DataNode n	Replica present	$d_{f_3}(n)$ accesses	Role in the move
n_1	Yes	50	Candidate source (underused replica)
n_2	Yes	200	Existing replica
n_3	Yes	150	Existing replica
n_4	No	200	Candidate destination (high remote demand)

The Locality manager selects the source node s^* as the replica that serves the fewest accesses in the current window and the destination candidate d^* as the node without a replica that issues the most requests for the file. In this example, $s^* = n_1$ and $d^* = n_4$. The projected gain in locality when moving one replica from n_1 to n_4 is

$$\Delta_{\text{loc}}(f_3; s^* \rightarrow d^*) = \frac{d_{f_3}(d^*) - d_{f_3}(s^*)}{\text{freq}(f_3)} = \frac{200 - 50}{600} = 0.25.$$

If this gain exceeds the configured threshold γ and all safety checks on rack diversity, capacity, node health, and move budgets pass, the relocation algorithm performs a two phase move. It creates a temporary extra replica of f_3 on n_4 , waits until the replica is in sync, then deletes the original replica on n_1 and updates the replica set. The replication factor stays equal to three, but future accesses from n_4 will be served locally instead of crossing the network.